```
$ curl -v -X 'PUT' -F 'file=@./meow.txt' 'http://localhost:8080/index.php'
> PUT /index.php HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.7.1
> Accept: */*
> Content-Length: 210
> Content-Type: multipart/form-data; boundary=-------Tb2Qb7aQRVJTEcBlTehHvR
...
string(245) "--------------------Tb2Qb7aQRVJTEcBlTehHvR
Content-Disposition: form-data; name="file"; filename="meow.txt"
Content-Type: text/plain

Jess is pretending she hasn't been fed. Again.

-------------------------------------Tb2Qb7aQRVJTEcBlTehHvR--
"
* Closing connection
```

In this case, we can see the raw sub-headers in our request body, because PHP isn't doing anything magical with the request data (remember PHP will only populate `$_FILES` and `$_POST` if the request method is POST). You'll need to do a little extra work with these types of requests to make sure the sub-headers, extra newlines, and boundary strings are stripped from the file data.

Building and Parsing URLs

Before we move on to looking at HTTP responses, let's look again at URLs. At the start of this section we learned that URLs are specially formatted strings that can be divided into five components (scheme, authority, path, and so on). When we looked at headers I gave a quick example of using PHP's sockets API (`file_get_contents()` and `stream_context_create()`) to make a request but simply used a string literal for the URL (supplied as the `filename:` argument to `file_get_contents()`). In reality you will likely need a lot more control over URL construction, and fortunately PHP provides some useful functions for doing just that.